Práctico 02 — Del triángulo a la malla

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

28 de agosto de 2026



Clínica: hacer fallar el triángulo

Pre requisitos

El programa que grafica un triángulo de color sólido funcionando sin errores.

Hacer fallar el programa

	Qué se comenta o se modifica
1	La línea que **habilita el atributo** del VAO
2	Un punto y coma del **vertex shader** — y no mirar el log
3	El vértice de arriba: 0,5 → 1,5

Predecir lo que va a suceder

Antes de correr, escribir en papel qué se espera ver. Una predicción equivocada es el único resultado que enseña algo.

Al terminar cada una, dejar el código andando otra vez.

Análisis de los resultados

1. **Atributo apagado** $\rightarrow$ pantalla "negra" sin ningún error. Los tres vértices reciben $(0, 0, 0, 1)$ y el triángulo se degenera en un punto.

   La checklist no puede detectarla: se determina "manualmente" sobre el código.

2. **Shader roto, log no consultado** $\rightarrow$ pantalla "negra", sin mensaje, y el programa **compiló perfecto**.

   El shader es un string hasta que la aplicación se ejecuta: el compilador se encuentra en el driver.

3. **Vértice en** 1,5 $\rightarrow$ el triángulo no desaparece: aparece **recortado** por el borde superior.

   Los vértices se escriben "a mano" dentro del rango $[-1; 1]$.

Tres fallas distintas. **Ninguna se diagnostica tanteando.**

¿Cuántos vértices tiene un cubo?

Sin pensarlo mucho.

Un *vértice* es más que una coordenada

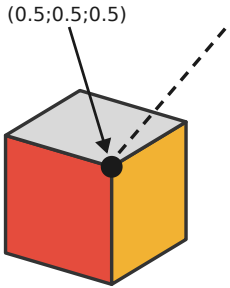
- Una **coordenada** es una posición en el espacio: tres valores.
- Un **vértice** (en el contexto de OpenGL) es el conjunto de *dato* que recibe el vertex shader en cada llamada: una **tupla completa de atributos**.

```
struct Vertex
{
    float px, py, pz;   // posición
    float r,  g,  b;    // color
};
```

En el VBO se pueden guardar **tuplas**, una atrás de la otra.

tupla o **upla**: lista (secuencia) ordenada y finita de elementos

Atributos en un vértice geométrico del cubo



tres *vértices* distintos

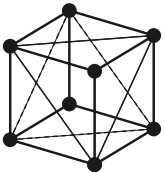
(0.5, 0.5, 0.5)	cara frente
(0.5, 0.5, 0.5)	cara derecha
(0.5, 0.5, 0.5)	cara arriba

misma posición, distinto color

Cada vértice geométrico se comparte en **tres caras** y cada cara tiene un color diferente.

Vértices necesarios

- Cómo las caras del cubo se van a dibujar con triángulos y hay vértices compartidos → se pueden reducir la cantidad de vértices a enviar al GPU



4 vértices (geom.) × 6 caras	**24**	vértices únicos
6 caras × 2 triángulos × 3 vg/triang	**36**	índices

- **Entre caras no se puede reducir nada**: los colores son diferentes.
- En cambio los dos triángulos que forman una cara comparten la diagonal, así es que se puede dibujar con **4 vértices y 6 índices** en vez de 6 vértices.
- Sin usar índices, el mismo cubo requiere **36 vértices**.

24 tuplas × 24 bytes = 576 bytes, más 144 de índices, total 720 bytes. Sin índices: 864 bytes.

La regla general para todas las mallas poligonales

Un vértice se comparte **si y sólo si** coinciden *todos* sus atributos.

- Es un **permiso, no una obligación**: se *pueden* fundir si coinciden. A veces conviene no hacerlo.
- Si los *vértices* del cubo tuvieran sólo posición —ni color, ni normal, etc.— entonces sí serían 8 vértices y 36 índices. La respuesta depende de qué información lleva la tupla.

La misma regla contesta el cilindro, el cono y el terreno. Vuelve en el bloque de papel.

Datos para una malla indexada: dos listas

```
struct MeshData
{
    std::vector<Vertex>       vertices;   // conjunto mínimo requerido
    std::vector<unsigned int> indices;    // orden de lectura para formar triángulos
};
```

- **Vértices:** las tuplas que se requieren para modelar el objeto. Cada una aparece una sola vez.
- **Índices:** en qué orden hay que leerlas para formar los triángulos. Cada tres índices, un triángulo.

Todavía no hay nada de OpenGL acá: es un **dato**, no un recurso.

Element Buffer Object (EBO)

- El EBO es **un buffer más**: bytes sin tipo, igual que el VBO.
- También se suele denominar *Index Buffer Object* (IBO)
- EL sentido del contenido ("índices") se lo da la ranura (slot) dedicada del VAO al que se vincula.
  - Los VBO se conectan por puntos de enlace **numerados** (`glVertexArrayVertexBuffer(vao, 0, ...)`).
  - El EBO se conecta por una ranura **única** (sin índice).
- Esa ranura es **parte del estado del VAO**.

```
glCreateBuffers(1, &ebo_);
glNamedBufferData(ebo_, indices.size() * sizeof(unsigned int),
                  indices.data(), GL_STATIC_DRAW);

glVertexArrayElementBuffer(vao_, ebo_);   // <- se conecta al VAO
```

Dibujar con índices: glDrawElements

```
// antes (triangulo):  se recorren los vértices en orden
glDrawArrays(GL_TRIANGLES, 0, 3);

// ahora (cubo): se recorren los ÍNDICES, y a partir de ellos eligen los vértices
glDrawElements(GL_TRIANGLES, 36, GL_UNSIGNED_INT, nullptr);
```

- El segundo argumento en `glDrawElements` indica cantidad de **índices** (no vértices).
- El `nullptr` del final no es un puntero a memoria de la aplicación: es un **offset dentro del EBO**. Es herencia de cuando OpenGL sí leía de la memoria del programa.

Configuración de los atributos

- Los datos por vértice se cargan **entrelazado** en buffer
- Hay que configurar los atributos para poder leerlos

```
glVertexArrayVertexBuffer(vao_, 0, vbo_, 0, sizeof(Vertex));   // stride

glVertexArrayAttribFormat(vao_, 0, 3, GL_FLOAT, GL_FALSE, offsetof(Vertex, px));
glVertexArrayAttribBinding(vao_, 0, 0);         // atributo 0 <- punto de enlace 0
glEnableVertexArrayAttrib(vao_, 0);

glVertexArrayAttribFormat(vao_, 1, 3, GL_FLOAT, GL_FALSE, offsetof(Vertex, r));
glVertexArrayAttribBinding(vao_, 1, 0);         // atributo 1 <- punto de enlace 0
glEnableVertexArrayAttrib(vao_, 1);
```

- Un solo buffer, un único punto de enlace, **dos atributos**. Lo que los separa es el *offset* dentro de la tupla, no el buffer.

`offsetof()`: returns the offset in bytes of the specified member from the beginning of its parent data structure

Shaders: lo nuevo que hay que agregar

Vertex Shader

```
#version 460 core

layout (location = 0) in  vec3 aPos;           // entra desde la config del VAO
layout (location = 1) in  vec3 aColor;

out vec3 vColor;                               // sale hacia el fragment shader

const mat3 kTransform = mat3( /* ... */ );     // constante, se calcula al compilar

void main()
{
    vColor = aColor;
    gl_Position = vec4(kTransform * aPos, 1.0);
}
```

Nuevo hoy: **un segundo atributo de entrada**, una variable que se pasa entre etapas del pipeline (`out`) y una matriz `mat3` con su multiplicación.

Los nueve números de `kTransform` **se dan escritos**: caja negra, y están al final.

Shaders: lo nuevo que hay que agregar

Fragment Shader

```
#version 460 core

in vec3 vColor;          // atributo que entra, se lee en vertex shader

out vec4 FragColor;

void main()
{
    FragColor = vec4(vColor, 1.0);
}
```

Nuevo hoy: **una variable de entrada** (`in`) que viene del pipeline de la GPU. El tipo y el nombre tienen que coincidir exactamente con la declaración en la etapa programable de la que sale (vertex shader en el ejemplo)

Programa del cubo y el encapsulado

Los **mismos cuatro movimientos** del triángulo. Se pueden leer uno al lado del otro.

	14-ago (triángulo)	**hoy (cubo)**
1. decidir los datos	3 vértices a mano	`primitives::cube()`: 24 y 36
2. bytes a la GPU	llamadas sueltas en `main`	`Mesh::load()`
3. armar el programa	dos *strings* en el `.cpp`	archivos `.vs`/`.fs` + `ResourceManager`
4. dar la orden	`glDrawArrays`	`glDrawElements`

- El `main.cpp` de hoy tiene **menos** líneas de OpenGL que el del triángulo, y dibuja más.
- No posee ningún identificador de OpenGL: no hay un solo `glDelete*`. Los dueños son `Mesh` y `Shader`.
- Lo que cambió no es OpenGL: cambió **quién es dueño de qué**.

Éste es el programa que hay que armar **para el viernes**.

Dos memorias y propiedad (ownership)

¿Quién es el **dueño** del VAO y los *buffer objects*

y qué pasa cuando ese dueño **desaparece**?

- El VAO, el VBO y el EBO se gestionan por medio de *nombres* representado por **enteros**. No son instancias de objetos (del lado del CPU).
- El objeto real se encuentra en la GPU.
- Copiar el entero no copia el objeto.
- Perder el entero no libera el objeto: lo deja huérfano hasta que se cierra el contexto.

Interfaz de Mesh: propiedad y ciclo de vida

Propuesta: Gestionar la creación, propiedad y liberación de los objetos de OpenGL asociados a la memoria para enviar datos al GPU (VAO y buffer objects)

```
class Mesh
{
public:
    Mesh() = default;
    ~Mesh();                                    // libera lo que posee

    Mesh(const Mesh&) = delete;                 // copiar rompe el dueño único
    Mesh& operator=(const Mesh&) = delete;

    Mesh(Mesh&& other) noexcept;                // mover: se traspasa la propiedad
    Mesh& operator=(Mesh&& other) noexcept;

    void load(const MeshData& data);
    void clear(void);

    unsigned int vao(void) const  { return vao_; }
    int count(void) const         { return count_; }
private:
    unsigned int vao_ {0U}, vbo_ {0U}, ebo_ {0U};
    int count_ {0};
};
```

Mesh: movimiento y limpieza

Estado del objeto movido: queda en cero

```
Mesh::Mesh(Mesh&& other) noexcept :
    vao_(other.vao_), vbo_(other.vbo_), ebo_(other.ebo_), count_(other.count_)
{
    other.vao_ = other.vbo_ = other.ebo_ = 0U;   // se anula el origen
    other.count_ = 0;
}
```

Limpieza:

```
void Mesh::clear(void)
{
    glDeleteVertexArrays(1, &vao_);   // glDelete*(0) es legal: no hace nada
    glDeleteBuffers(1, &vbo_);
    glDeleteBuffers(1, &ebo_);
    vao_ = vbo_ = ebo_ = 0U;
    count_ = 0;
}
```

`clear()` se puede llamar dos veces y no hace falta ninguna bandera extra.

Interfaz de `Shader`: propiedad y ciclo de vida

Propuesta: lo mismo, sobre otro recurso —el **programa** linkeado.

```
class Shader
{
public:
    Shader() = default;
    ~Shader();                                  // libera el programa
    Shader(const Shader&) = delete;             // las MISMAS tres decisiones
    Shader& operator=(const Shader&) = delete;
    Shader(Shader&& other) noexcept;
    Shader& operator=(Shader&& other) noexcept;

    bool compile_from_source(const std::string& vs, const std::string& fs);
    void use(void) const;
    void clear(void);

    unsigned int id(void) const  { return id_; }
private:
    unsigned int id_ {0U};                      // un entero, no cuatro
};
```

```
void Shader::clear(void)
{
    glDeleteProgram(id_);   // glDeleteProgram(0) es legal: no hace nada
    id_ = 0U;
}
```

Las mismas tres decisiones, sobre otro recurso.

Gestión de shaders

Quién lee el disco y quién compila

```
ResourceManager recursos(assets_root);      // C++ puro: no sabe qué es un shader

const ShaderSource& fuente =
    recursos.load_shader_source("solid", "shaders/solid.vs",
                                         "shaders/solid.fs");

Shader shader;
shader.compile_from_source(fuente.vs, fuente.fs);   // OpenGL: no abre archivos
```

- `ResourceManager`: gestiona la lectura desde disco, *no lee dos veces* y guarda copia del código del shader.
- `Shader`: compila, linkea y libera el programa (es el dueño).
- Más adelante `Shader` va a tener que incorporar otras funcionalidades
- **Ninguno hace el trabajo del otro.** Ése es todo el criterio.

Trampa: el orden de destrucción

```
{
    Mesh cubo;
    Shader shader;
    // ... el render loop ...
}                          // <- acá se ejecutan ~Mesh y ~Shader, con el contexto activo

glfwDestroyWindow(window);
glfwTerminate();
```

- Los dos destructores terminan en un `glDelete*` —`glDeleteVertexArrays` y `glDeleteProgram`, las dos filminas anteriores—: tienen que correr **antes** de `glfwTerminate()`.
- Si `Mesh` vive hasta el final de `main`, su destructor corre sin contexto. A veces no pasa nada. A veces es un segfault al cerrar, de los que aparecen "a veces".

Es la contracara de liberar automáticamente: hay que cuidar **cuándo**.

Actividad de aula: de a dos, a mano: 10 minutos

Un **cilindro** de 8 gajos, con tapas, con un color por gajo y otro por tapa.

1. ¿Cuántos vértices tiene la **superficie lateral**? ¿Y cuántos índices?
2. La **costura** —donde el último gajo se encuentra con el primero— ¿comparte vértices o no?
3. Las **tapas**: ¿el borde de la tapa comparte vértices con el borde del lateral?

La única regla que hace falta

Se comparte un vértice **si y sólo si** coinciden todos sus atributos.

Práctico 02 — el cubo indexado

- **Implementar las clases** `Mesh` y `Shader`
- **Escribir el módulo** `primitives`, que genera la malla
- **Armar el cubo** sobre el template usando `Mesh`, `Shader`, `primitives` y el `ResourceManager` propio.

La próxima clase vamos a necesitar todo eso porque:

- Se aplican transformaciones
- Los vértices dejan de introducirse a mano
- Aparecen nuevas primitivas

Si tiene inconvenientes, pueden hacer consulta durante la semana.

El enunciado completo —**Práctico 02**— se cuelga del aula.

El resultado al que hay que llegar



- 24 vértices, 36 índices.
- Un color por cara, sin iluminación.
- Tres caras visibles: las otras tres las descarta el **test de profundidad**.

Las dos cajas negras de hoy

Dos cosas se usan hoy **sin explicar**. Las dos tienen fecha.

`solid.vs` — rota el cubo. Vence el **04-sep** (transformaciones):

```
const mat3 kRotacionFija = mat3(     // ojo: GLSL recibe COLUMNAS
    vec3( 0.3686, -0.1454, -0.3119),
    vec3( 0.0000,  0.5438, -0.2536),
    vec3(-0.2581, -0.2077, -0.4454));
```

`main.cpp` — descarta lo que está más lejos. Vence el **09-oct** (Unidad VIII):

```
glEnable(GL_DEPTH_TEST);                                  // una vez, al iniciar
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);       // cada cuadro: DOS bits
```

Sin la primera, el cubo se vería de frente: **un cuadrado**.

Sin la segunda, las caras se pisan en el orden en que se dibujan.

Los dos archivos quedan en el aula: **no hay que tipear los nueve números**.

Quiero dibujar **dos** cubos.

¿Cuánta memoria de GPU cuesta el segundo?